Project FAQ

Comprehensive Frequently Asked Questions about this project.

General

Q1. What is this project?

A: A lightweight mini-ERP example combining a Node.js backend and a React frontend for demonstration and interviews.

Q2. What languages and frameworks are used?

A: Backend: Node.js/Express; Frontend: React (Vite); Docs and scripts in Python where useful.

Q3. Where is the main application code located?

A: Backend lives in the `backend/` folder and the frontend in the `frontend/` folder.

Setup

Q1. How do I install dependencies for the backend?

A: Run `npm install` inside the `backend/` directory.

Q2. How do I install dependencies for the frontend?

A: Run `npm install` inside the `frontend/` directory.

Q3. Are there any environment variables required?

A: The project reads DB and auth config from environment variables; check `src/config` for names and defaults.

Backend

Q1. What is the backend entrypoint?

A: The backend uses `server.js`/`app.js` at the repository `backend/` root to start the Express app.

Q2. How are controllers organized?

A: Controllers are in `backend/src/controllers` handling request logic and delegating to services.

Q3. Where are middleware functions located?

A: Middleware is in `backend/src/middleware`, including auth and error handling middleware.

Frontend

Q1. What frontend tooling is used?

A: Vite for bundling and fast dev server; React for UI components.

Q2. Where are API calls defined?

A: API helper functions are in `frontend/src/api`, with an `axios` instance wrapper.

Q3. How to run the frontend in development?

A: From `frontend/` run `npm run dev` (Vite) to start the dev server.

API

Q1. Where are the API routes defined?

A: Backend routes are in `backend/src/routes` and mounted from `routes/index.js`.

Q2. What auth method is used for API requests?

A: Token-based authentication (JWT) via auth middleware protecting routes.

Q3. Is there an OpenAPI or Postman collection?

A: Yes — see `backend/docs/mini_erp_phase_2.postman_collection.json` and `openapi.json`.

Database

Q1. What database is expected?

A: The code is written to be DB-agnostic, but typically a relational DB (Postgres/MySQL) is used; check `src/config/db.js`.

Q2. Where are models defined?

A: Models are in `backend/src/models` representing customers, products, inventory, sales, and users.

Q3. How to seed sample data?

A: Run the seeder script in `backend/seeders/seed.js` after setting DB connection parameters.

Security

Q1. How is password storage handled?

A: Passwords should be hashed (bcrypt recommended). Ensure `user` service uses secure hashing before saving.

Q2. How to add role-based access control?

A: RBAC middleware exists in `src/middleware/rbac.middleware.js` — attach it to protected routes and configure roles.

Q3. What should I check for production security?

A: Use HTTPS, secure JWT secret management, proper CORS, rate limiting, and avoid exposing stack traces.

Testing

Q1. Are there automated tests?

A: This repository does not include a full test suite by default — add Jest/Mocha tests under a `tests/` folder.

Q2. How to run manual API tests?

A: Use the included Postman collection in `backend/docs` or curl against the running server.

Q3. What test data is recommended?

A: Seed a small dataset with `seeders/seed.js` to exercise endpoints for customers, products, and sales.

Deployment

Q1. What are the deployment options?

A: Containerize with Docker, deploy to cloud VMs, or use PaaS like Heroku/Render; serve frontend statically or from CDN.

Q2. How to prepare for production?

A: Set NODE_ENV=production, configure DB credentials securely, enable logging to files/aggregation, and run health checks.

Q3. Any build steps for frontend?

A: Run `npm run build` in `frontend/` to produce static assets for hosting.

Maintenance

Q1. How to rotate secrets and credentials?

A: Store secrets in a secrets manager; update environment variables and restart services; automate rotation where possible.

Q2. How to backup the database?

A: Use DB-native backup tooling (pg_dump/mysqldump) and schedule regular snapshots offsite.

Q3. How to update dependencies safely?

A: Use a staging environment to test dependency updates, run tests, and deploy gradually.